Context Contagion in Real Transformers: A Progress Report

Dan Shumow*

November 2026 (revised from the June 2026 progress report)

Revision note. The June 2026 draft of this report was reviewed in November 2026 and found to overstate several results: the "condensation knee" for multi-token payloads was the grid floor; the at-scale trust-versus-usefulness test had repeating "non-repetitive" cells; the detector's AUC rested on three negatives with no baseline; one of two firebreak runs went unreported; no reproduction number had been computed. Every affected experiment was redesigned and rerun with committed per-trial data (GPT-2-small locally; 355M–7B on cloud GPUs, 60 chains per cell where survival is reported). This version states only what those reruns support. The record of every run is `results_log.md`; the review and its work plan are `review_remediation_plan.md`; the pod runs are audited in `TIER2_AUDIT_NOTES.md`.

Abstract

Two companion analyses [1, 2] conjectured, from an exactly solvable Markov toy model, that in-context exploitability tracks *trust* rather than usefulness, and that repeated token strings can be planted in a context, hijack generation, and spread between contexts as a worm. We test these conjectures on real transformers (124M–7B, five families, base and instruct) with controls and committed per-trial data. *What holds.* A novel span of three or more tokens is reproduced after a single prior occurrence in most settings, and ablating the top 5% of heads by induction score collapses this at every scale, while a fixed head count produces a false null. A repeated span hijacks generation only when it *ends* the context: any coherent text after it, whether a task query or unrelated prose of the same length, removes the hijack on three instruct models, so the retrieval-augmented threat is a recency gate, not a matter of task-following. *What does not hold.* There is no reproduction number above one, no critical payload length, and no tokenizer firebreak; surviving chains are locked repetition loops, and the family that loops most (Qwen2.5 base, at every size) is a model trait, not a scale law. A detector built on induction attention is no better than counting repeated n-grams.

1 Scope and summary

The conceptual framing, the exact toy results, and the falsifiable conjectures live in the companion write-ups [1, 2]; we recap only what is needed. The threat is concrete—context poisoning and self-replicating prompts already target real retrieval-augmented and agent pipelines [6, 5]—and the toy substrate is a from-scratch analogue of the parametric-versus-in-context split studied by Bietti et al. [4]. The experiment plan (claims, controls, statistics, phasing) is the repository's

*Work-in-progress write-up of the open-source project `transformer-context-contagion`, the empirical follow-on to the toy-model analyses `trust-is-the-attack-surface`. All numbers and figures are reproduced by the `experiments/` scripts in that repository; the running log is `results_log.md`.

`transformer_validation_plan.md`; every run, including the ones this version supersedes, is in `results_log.md`.

In order of what we think matters:

1. **Scale-matched ablation.** The induction-head attribution holds from 124M to 7B only when the ablated set is a *fraction* of heads; a fixed count of eight heads gives a false null at 1.5B. (§10.1)
2. **The RAG bound is a recency gate.** On three instruct models, a repeated span that ends the context is copied in 85–100% of trials; the same span followed by a task query, an instruction, or 12–21 tokens of unrelated prose is copied in 0–3%, with or without a chat template. The task is incidental. (§16)
3. **One occurrence suffices.** For spans of ≥ 3 tokens the one-shot copy rate is $\geq \frac{1}{2}$ in 36 of 48 model–length cells at 355M–7B; instruction tuning raises the count needed but does not remove copying. The "condensation knee" of the first draft was the grid floor. (§9)
4. **Poisonability follows repetition, not usefulness**, on three instruct models with a real task—but the measurement is the model being in repetition mode, and "trust" is a label for that axis, not a separately measured quantity. (§16)
5. **No worm.** No per-hop reproduction ratio above one in any of 30 model–length cells at 60 chains each; survivors are locked loops; survival is flat in payload length; the tokenizer firebreak does not replicate at n=30. Qwen2.5 base models loop far more than Pythia, Llama, Gemma or GPT-2 at every size, which is a family trait. (§16)
6. **The detector is an n-gram counter.** Induction attention and a repeated-trigram count both separate poisoned from clean at AUC 1.0; the count has fewer false positives on benign repetition. (§14)

Experiment	Final status	Result (section)
E1.1–E1.5 GPT-2-small	done, reruns committed	one-shot rate 0.20 at p=3, 0.60 at p=5; cue-dependent; N not length; distance-sensitive (§5–7)
E1.3/E1.4 at scale	done	$\geq \frac{1}{2}$ one-shot in 36/48 cells; instruct models need more occurrences (§9)
E2 attribution	done, 124M–7B	induction ablation -64 to -98% vs random ≤ -28% near threshold; fixed count fails (§10, 10.1)
E4 trust vs usefulness	redesigned, rerun	$\mathrm{corr}(P,T)$ $+0.15$ to $+0.38$, $\mathrm{corr}(P,U)$ -0.18 to $+0.07$ on three instruct models (§16)
E5 RAG hijack	rerun with controls	recency gate: raw 85–100% hijacked, anything after the span 0–3% (§16)
E6 worm / firebreak	rerun at n=60 / 30	withdrawn: no $R > 1$, no critical length, no firebreak; Qwen-family looping (§16)
E7 detector	rerun with baseline	no better than a trigram count (§14)
E8 substitution / capstone	rerun control	induction-carried at $m \geq 8$, smaller effect; instruct models self-extract (§15)

Table 1: Final status of the experimental program. Stage numbering follows `transformer_validation_plan.md`.

2 Claims under test

Each maps a toy object to a transformer-measurable quantity; see the plan for the full list and the experiments E1–E6.

	Toy result	Real-transformer prediction
C1	condensation knee	a repeated OOD string is regenerated past a sharp threshold in N
C2	induction is the engine	reproduction is carried by induction heads (copy-on-match)
C3	length penalty vanishes under induction	knee location roughly flat or *decreasing* in length
C4	exploitability tracks trust, not usefulness	repetitive contexts most poisonable, $\approx$ independent of benefit
C5	delivery = entry $\times$ lock-in	a planted span hijacks generation in a long context
C6	worm with $R_0 \approx T/(p\,k_{\text{thresh}})$	output fed back propagates; short strings spread, long die

Table 2: The conjecture, claim by claim, and its outcome. **C1/C3** hold in the weak form "one prior occurrence suffices for spans of ≥ 3 tokens"; the sharp threshold and its length-decrease are a grid artifact for multi-token payloads (§5, §9). **C2** holds behaviorally and mechanistically at every scale tested (§6, §10, §10.1). **C4** holds in direction on three instruct models, as a repetition effect (§16). **C5** holds as entry $\times$ lock-in on a base model and reduces to a recency gate on instruct models (§12, §16). **C6** does not hold: no reproduction number above one, no critical length, no firebreak (§13, §16).

3 Related work, and our delta

Every ingredient here is known; the contribution is the *framing* that connects them and makes the connection measurable, plus the at-scale tests of that framing—some of which falsify its own comforting predictions.

Induction heads. Olsson et al. [3] identified induction heads as the in-context copy circuit ("saw AB, now see $A\rightarrow$ predict B"), and Bietti et al. [4] gave a memory-viewpoint account of how such copying is learned. We do *not* claim to discover this mechanism. Our delta is to measure it as a *security* quantity: for an *injected, out-of-distribution* span of three or more tokens, one prior occurrence suffices for reproduction more often than not across 355M–7B (C1/C3 in the only form the data support), and it is the *same* circuit that carries that reproduction, which we establish causally by ablation from 124M to 7B across three families (C2, §10.1)—with the methodological caveat that the ablated set must scale with the model or the attribution fails. The mechanism is borrowed; that it is the engine of context-poisoning *persistence* is the claim.

Indirect prompt injection. Greshake et al. [6] established that retrieved / tool-supplied content can compromise LLM-integrated applications. That work characterizes the *threat*; we add a *mechanistic* account of one axis of it—why a *repeated* payload persists and gets re-emitted—and two bounds on it. First, what predicts poisonability is *repetition*, decoupled from content usefulness on three instruct models with a real task ($\text{corr}(P, \text{useful})$ from -0.18 to $+0.07$ vs. $\text{corr}(P, \text{repetition})$

from +0.15 to +0.38, C4, §16); the "structural" detector this motivates turns out to be a repeated-n-gram counter (§14). Second, the hijack is gated by *recency*, not by the task: a benign OOD span copied in 85–100% of raw continuations is copied in 0–3% once *anything* coherent follows it, with or without a chat template (C5, §16). So the copy-dynamics account applies to the echo of a repeated span that ends the context, and the residual RAG threat of a *semantically plausible* planted instruction is a distinct problem we do not claim to cover.

GenAI worms. Cohen et al. [5] demonstrated self-propagating worms in GenAI-powered application pipelines (e.g. via RAG/embedding stores). We study the copy dynamics *underneath* such propagation on the models themselves: whether a payload survives serial passage, how fast an unlocked chain decays, and whether re-tokenization attenuates it. With 60 chains per cell on six models (§16) there is no reproduction number above one anywhere; what differs between models is how readily each *locks* into a payload loop, and that is a family trait (Qwen2.5 base at 0.5B–7B) rather than a function of scale. Any "GenAI worm" built on repetition therefore inherits the epidemiology of a degeneration bug, not of an infection.

Repetition and degeneration. Holtzman et al. [8] documented that sampled and greedy decoding from language models falls into repetition loops; Xu et al. [9] studied why and proposed training-time mitigations. The "endemic" chains in §16, with a flat viral load equal to the generation budget divided by the payload length, are this phenomenon seeded deliberately. Any claim that the worm "gets worse at scale" has to be made against this baseline: it is a claim about the propensity of larger models to enter and stay in such a loop. McDougall et al. [10] identified *copy-suppression* heads that actively reduce the probability of repeating a token just seen; this is the natural account of why single-token payloads (p=1) fail to lock in most families here while $\geq$ 3-token spans lock after one occurrence (§9). Jelassi et al. [11] measured transformers' capacity to copy long strings from context, the ability our length-robustness result (C3) rides on.

In-context repetition as an attack. The closest security-relevant prior is many-shot jailbreaking [12]: repeating demonstrations in context shifts a chat model's behavior with a power law in the number of shots, which is in-context "trust" accumulated by repetition in exactly the sense of C4. Nasr et al. [13] showed that asking a model to repeat a single token indefinitely makes it diverge and emit memorized training data—a repeated-token attack whose first stage is the p=1 frequency regime of §6. Liu et al. [14] characterized the position dependence of in-context use ("lost in the middle"), which is the recency profile that E1.5 and E5 measure for a planted span. Our delta with respect to all three is narrower than the first draft implied: a controlled measurement of the copy mechanism's one-shot rate and placement dependence for a *meaningless* span, with the mechanism attributed by ablation, and a bound showing that the hijack is a recency gate.

Toy companions. The two companion analyses [1, 2] work all of this out *exactly* on a global-plus-cache Markov model and state each result as a conjecture for real transformers. This paper is the real-transformer test. Of the frame's three non-obvious predictions, the length-robustness sign (C3) holds in the weak form "one occurrence suffices for $p \geq 3$"; trust $\perp$ usefulness (C4) holds in direction on three instruct models as a repetition effect; and the tokenizer firebreak (C6) does not hold. The toy's reproduction number $R_0 \approx T/(p\,k_{\text{thresh}})$, and the critical length it implies, are absent from the transformer data (§16).

4 Method

Payloads. A *payload* is a short cyclic string S of token ids the model would essentially never emit on its own, so any reproduction is the copy mechanism, not the model's taste—the analogue of the toy's rainbow / de Bruijn quines. Payloads are built at the *token* level (length p is in tokens) from a pool of rare, round-trip-stable single tokens, as distinct-token ("rainbow") cycles. We use R independent random payloads per length and average, with Wilson confidence intervals.

Scoring. Reproduction is scored at the level of *transitions*, not positions: a single slip only phase-shifts a positional comparison and tanks all later positions even while the model is faithfully reproducing. Given S and a generated continuation, the per-step fidelity is the fraction of steps obeying S's successor rule given the realized window; a continuation *reproduces* S if that fidelity exceeds $\frac{1}{2}$. P(reproduce) is the fraction of sampled continuations that do.

Controls. Three controls separate genuine pattern-copying from artifacts: (i) a *no-payload* (N=0) control—prime at the same start token but never show the payload—measures spontaneous emission, which must be ≈ 0; (ii) a *scrambled* control—the same tokens in shuffled order, destroying the repeated n-gram while preserving token frequencies; (iii) a *cue* control (§6) contrasting a matching with a non-matching generation prompt.

Setup. Results below are GPT-2-small (124M) [7], on an Apple-Silicon laptop (MPS/Metal backend), `torch 2.8`, `transformers 4.57`, sampled at temperature 1.0. This is a development-scale environment: it establishes the phenomena on one small model; the size sweep (E1.3) needs more compute.

5 E1.1: the condensation knee

We sweep the repetition count N and the string length p, estimating P(reproduce) over R=4 payloads $\times$10 sampled continuations per cell, with the scrambled and no-payload controls and a bootstrap interval on the knee location N^* (the interpolated P=$\frac{1}{2}$ crossing).

p	knee N^* (95% CI)	width	scrambled P @ N=32	no-payload P
1	7.2 (5.5–10.3)	7.8	**0.97**	0.00
2	4.3 (1.7–6.2)	5.2	**0.70**	0.00
3	3.0 (2.6–3.6)	1.6	0.05	0.00
5	1.9 (1.7–4.8)	3.2	0.00	0.00

Table 3: E1.1 on GPT-2-small. The knee N^* decreases with length; the no-payload control is 0 throughout; the scrambled control is near 0 for $p \geq 3$ but *not* for short p (see text).

C1 (the knee exists): confirmed. Every length shows a clear threshold in N (Figure 1), and the no-payload control is 0.00 everywhere (Table 3)—the payloads are genuinely out-of-distribution, so reproduction is caused by the repetitions, not the model's prior.

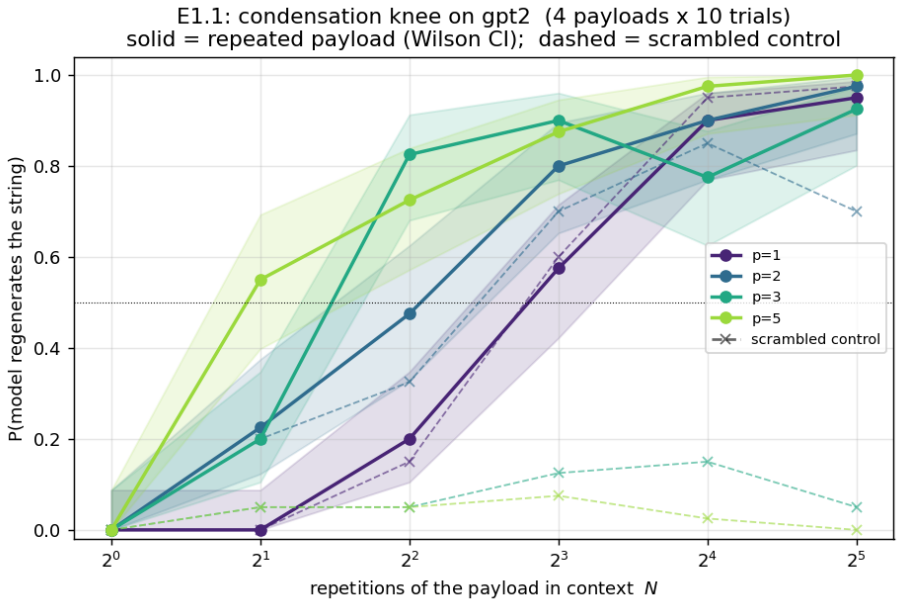


Figure 1: P(reproduce) vs. repetition count N on GPT-2-small, per string length (solid, with Wilson bands); dashed is the scrambled control. A clear knee in N exists for every length. Reproduced by `experiments/e1_condensation_knee.py`.

C3 (length robustness): holds, in a weaker form than first stated. N^* *decreases* with length, $7.2 \to 4.3 \to 3.0 \to 1.9$ (Figure 2): longer strings are *easier* to plant, the opposite of the fixed-order count cache's brittleness ladder [2]. But the p=5 value sits in the first grid interval: at N=1 the cyclic transition $S[-1] \to S[0]$ has never occurred, so copying it is impossible except by chance, and at N=2 it has occurred once. Any copy mechanism therefore jumps between N=1 and N=2, and an interpolated $N^* \in (1, 2]$ says only that the one-shot copy rate $P(\text{reproduce} \mid N{=}2)$ exceeds $\frac{1}{2}$ ($N^* = 1 + \frac{1}{2P(N=2)}$ when $P(N{=}1){=}0$). The decrease from p=1 to p=3 is a real threshold change; from p=3 on, "N^*" is the one-shot rate in disguise. §9 makes this explicit. A 2026-11-03 rerun of this experiment with the per-N curves retained gives the one-shot rate directly on GPT-2-small (Wilson 95% intervals, 40 samples per cell): $P(N{=}2) = 0.00$, $0.28\,[0.16, 0.43]$, $0.20\,[0.11, 0.35]$ and $0.60\,[0.45, 0.74]$ at $p = 1, 2, 3, 5$, with $P(N{=}1) \leq 0.03$ everywhere. So on the small model only p=5 is at the floor; one occurrence is *not* enough at p=3 (the knee of 3.0–3.2 there is a real threshold), and GPT-2-small is the weak end of the at-scale range in §9.

The scrambled control exposed a length-dependent mechanism. For $p \geq 3$ the scrambled control stays near 0 while the payload locks: reproduction needs the *pattern*. But for $p \leq 2$ the scrambled control reproduces almost as well $(0.97, 0.70)$, because it is *degenerate* at short length—a single repeated token has no order to scramble, and a bigram re-forms by chance. This motivated E1.2, which separates copying from frequency directly.

6 E1.2: copy versus frequency

Is reproduction genuine copy-on-match (induction) or a frequency bias (the repeated tokens simply become likely)? We separate them behaviorally—no activation access. After streaming S a fixed

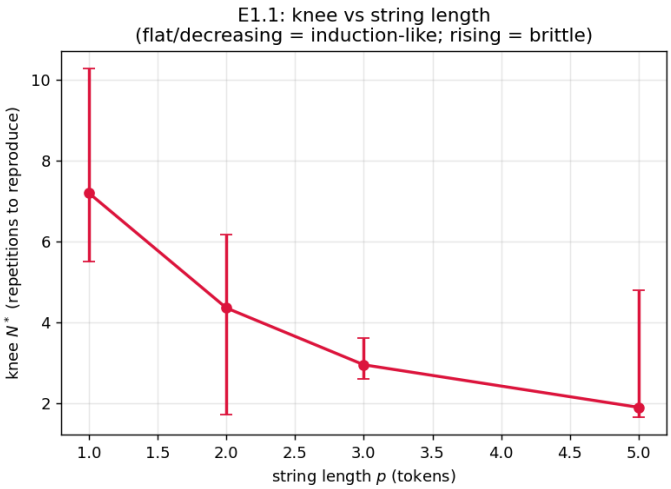


Figure 2: Knee N^* vs. string length (bootstrap CI). Decreasing N^* is the induction-like, length-robust behavior; a rising curve would indicate fixed-order brittleness. Values at or below 2 are at the grid floor (one prior occurrence already suffices) and should be read as a one-shot copy rate, not a threshold.

above-knee number of times N, we prime generation two ways: a *self-cue* at S's own tail (a matching cue), and a *neutral-cue* appending a fresh token $z \notin S$ (a non-matching cue), and compare $P(\text{reproduce } S)$. Induction is cue-triggered (the matching cue reproduces, the non-matching one should not); a frequency bias is cue-independent. The signature is the gap $P_{\text{self}} - P_{\text{neu}}$ vs. length.

p	P_{self}	P_{neu}	gap (N=16)	gap (N=32)
1	0.78–0.92	**0.26–0.29**	+0.51	+0.62
2	0.92–0.97	0.07–0.12	+0.79	+0.90
3	0.97–1.00	0.07–0.19	+0.90	+0.81
5	1.00	0.17–0.32	+0.83	+0.68
8	0.99–1.00	0.15–0.26	+0.85	+0.72

Table 4: E1.2 on GPT-2-small (R=6 payloads ×12 trials). The matching cue locks; the non-matching cue mostly fails. The gap is large at every length and smallest at p=1.

Reproduction is predominantly cue-dependent at every length (Table 4, Figure 3): the non-matching cue mostly fails ($P_{\text{neu}} \lesssim 0.3$) while the matching cue locks ($P_{\text{self}} \approx 0.8$–$1.0$), a gap of 0.5–0.9 well above zero. This is a behavioral induction [3] (copy-on-match) signature, established without interpretability tooling and consistent with C2.

This revises the E1.1 reading. Even p=1 is mostly copy (gap +0.5–0.6), not the pure-frequency regime E1.1's degenerate control suggested; it merely carries the *largest frequency tail* ($P_{\text{neu}} \approx 0.27$, versus ~ 0.1–0.2 for longer strings), since a single token has the biggest cue-independent pull and more distinct tokens dilute any one token's frequency. The length axis is therefore *copy-dominant-everywhere with a frequency tail largest at* p=1, not a clean regime switch.

Remark 1 (P_{neu} over-states frequency). After the non-matching cue the model can still re-enter

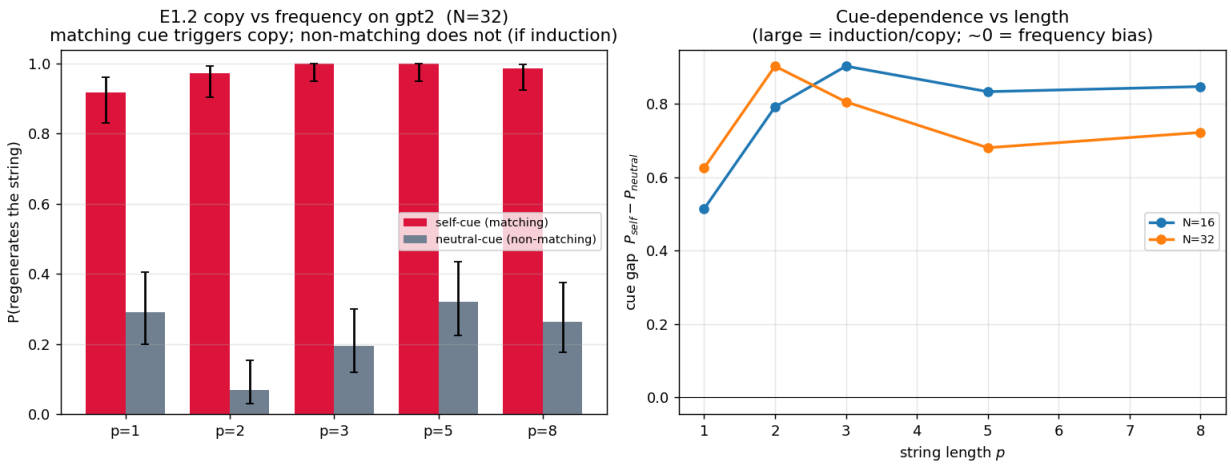


Figure 3: Copy vs. frequency. *Left:* matching (self) vs. non-matching (neutral) cue at N=32 per length, Wilson CIs. *Right:* the cue gap $P_{\text{self}} - P_{\text{neu}}$ vs. length; large everywhere (cue-triggered copy), smallest at p=1 (largest frequency tail). Reproduced by `experiments/e1_2_cue.py`.

S if it emits an S-token by chance and induction then completes the pattern (the toy's re-entry / self-healing). So P_{neu} is an *upper* bound on the pure-frequency component, and the true copy fraction is if anything larger than the gap suggests.

7 E1.5: is the knee an N effect or a context-length effect?

E1.1's context is $S \times N$, so the number of repetitions and the total context length grow together. E1.5 disentangles them. *Test 1* compares a *growing* context ($S \times N$) against a *fixed* total length (filler padded to a constant length) at each N. The two curves largely coincide (Figure 4, left): at fixed total length the threshold still rises sharply with N, so **the knee is predominantly an N effect**, not an artifact of longer context. A secondary *dilution* effect remains (padding at low N modestly lowers reproduction, strongest for longer p). *Test 2* holds N above the knee and inserts a gap of G neutral tokens between the payload and a re-presented matching cue. Reproduction **decays with distance**—gone by $G \approx 16$–32 tokens for $p \geq 3$ (Figure 4, right). So in GPT-2-small the copy is recency-modulated, not arbitrarily long-range; a payload buried deep in a long context is much weaker than one near the end. (The gap is filled with rare tokens, so part of the decay may be filler competition rather than pure distance; larger models with longer-range induction should push the limit out.)

8 E1.3-lite: cross-family replication, and an 8 GB wall

A size sweep over GPT-2 {small, medium, large} and Pythia {160M, 410M} was intended. The dominant outcome was a *hardware* one: on an 8 GB machine, gpt2-medium (355M) and larger thrash on swap (gpt2-large, 3 GB of weights, ran 47 wall-minutes using ~9 CPU minutes and completed nothing). Only GPT-2-small (124M) and Pythia-160M (162M) run cleanly, so a genuine size-scaling sweep is **deferred to more compute**. What did run shows the condensation knee and its length-robustness (N^* decreasing with p) in *both* model families (Figure 5), a modest cross-family generality check; the GPT-2 knees also replicate E1.1. Pythia-160M does not lock a single token

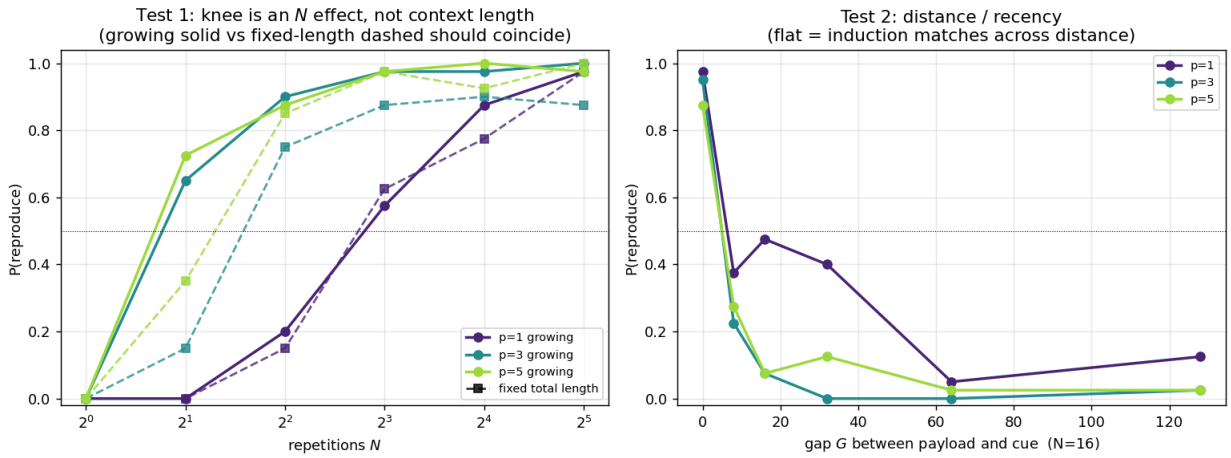


Figure 4: E1.5. *Left:* growing (solid) vs. fixed total length (dashed) – the knee persists at fixed length, so it is an N effect. *Right:* reproduction decays as the payload is separated from the cue by a gap of G neutral tokens (recency-modulated). Reproduced by `experiments/e1_5_context.py`.

within $N \leq 32$ (no knee at p=1), i.e. it is harder to frequency-drive than GPT-2, consistent with E1.2. The "knee vs. parameters" panel spans only two near-identical sizes and is *not* a scaling result.

9 At scale: the knee from 355M to 7B, base and chat (E1.3/E1.4)

The size sweep the 8 GB machine blocked (§8) was re-run on an Azure NC4as_T4_v3 (NVIDIA T4, 16 GB), in `float16`, across **twelve base models in four families spanning 355M–7B**—GPT-2 {medium, large, xl}, Pythia {410M, 1.4B, 2.8B}, Qwen2.5 {0.5B, 1.5B, 3B, 7B}, Llama-3.2 {1B, 3B}—and then on four **instruction-tuned** models (E1.4). Main condition only (controls validated on GPT-2-small); knee $N^*(p)$ for $p \in \{1, 3, 5, 8\}$, $N \in \{1, \ldots, 32\}$.

model	params	$N^*(p{=}1)$	p=3	p=5	p=8
GPT-2-medium	355M	5.1	2.7	1.8	1.7
GPT-2-large	774M	6.3	1.9	1.6	1.6
GPT-2-xl	1.5B	9.3	1.9	1.6	1.7
Pythia-410m	410M	–	2.4	1.7	1.9
Pythia-1.4b	1.4B	–	2.3	1.6	2.3
Pythia-2.8b	2.8B	–	2.3	1.5	1.8
Qwen2.5-0.5B	0.5B	–	1.8	1.5	1.7
Qwen2.5-1.5B	1.5B	–	1.8	1.9	1.6
Qwen2.5-3B	3B	–	1.8	1.7	1.7
Qwen2.5-7B	7B	–	1.5	1.6	1.6
Llama-3.2-1B	1B	–	1.9	2.7	1.9
Llama-3.2-3B	3B	–	1.7	1.9	1.6

Table 5: E1.3 size sweep (Track B, T4). "–" = no knee ($P < \frac{1}{2}$ for $N \leq 32$). The $p \geq 3$ knee is low ($N^* \approx 1.5$–2.7) and essentially size-independent.

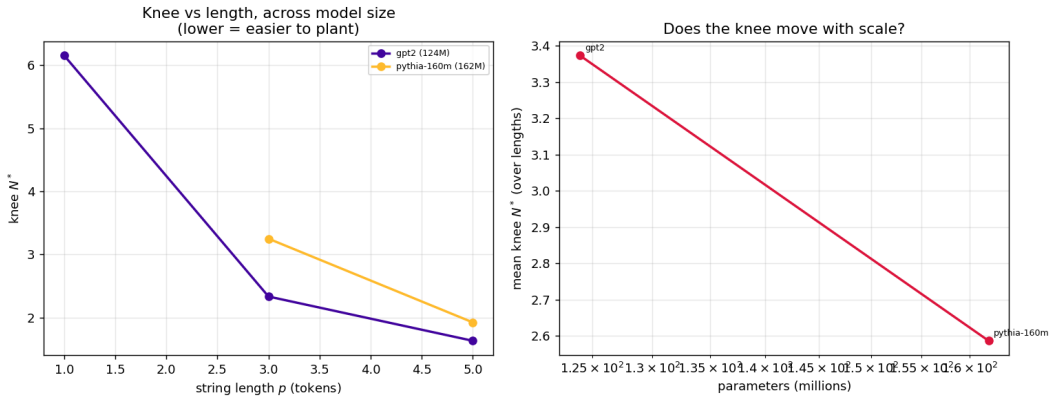


Figure 5: E1.3-lite. *Left:* the knee decreases with length in both GPT-2 and Pythia (cross-family replication). *Right:* two near-identical sizes – not a scaling result; a real sweep needs >8 GB. Reproduced by `experiments/e1_3_size_sweep.py`.

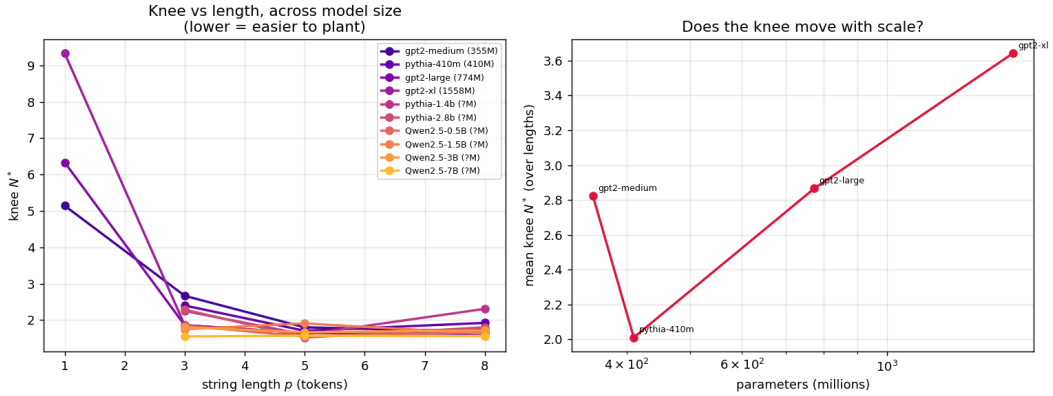


Figure 6: E1.3 at scale. *Left:* knee N^* vs. string length for each model—low and roughly flat for $p \geq 3$ at every size and in all four families. *Right:* mean knee vs. parameters—no downward trend for $p \geq 3$; the copy mechanism is already saturated by 355M. Reproduced by `experiments/e1_3_size_sweep.py`.

C3 holds across scale and family, as a one-shot rate. For $p \geq 3$, 30 of the 36 base-model cells in Table 5 have $N^* \leq 2$, i.e. the curve crosses $\frac{1}{2}$ inside the first grid interval: a novel span of ≥ 3 tokens is reproduced after *one* prior occurrence more often than not, at every size up to 7B and in all four families (Fig. 6). Inverting the interpolation (assuming $P(N{=}1){=}0$, which the GPT-2-small curves show) gives an implied one-shot copy rate with median 0.71 over the floor cells. The six cells above the floor (gpt2-medium p=3; Pythia-410m/1.4b/2.8b p=3; Pythia-1.4b p=8; Llama-3.2-1B p=5) need between one and three prior occurrences.

The "flat knee" is the grid floor, not a scaling result. The first draft read the size-independence of $N^*(p \geq 3)$ as "the copy mechanism is saturated by 355M." That reading is not supported: a quantity pinned to the first grid interval cannot show a trend. What the sweep does establish is that the one-shot rate is $\geq \frac{1}{2}$ nearly everywhere; whether it *rises* with scale within $[0.5, 1]$ is a question the stored knees cannot answer and the per-N curves (not retained by the

sweep script, now fixed) can. Size dependence proper lives in *single-token* lock-in: only the GPT-2 family locks p=1 within $N \leq 32$, and its knee *rises* with size (5.1 → 6.3 → 9.3). With E1.2 this says p=1 reproduction is a model-specific frequency-prior effect, consistent with copy suppression [10], while the $p \geq 3$ one-shot copy is universal.

E1.4: instruction-tuning does not immunize. The worry that chat models would refuse or summarize rather than copy did not materialize: all four ran clean and reproduced the planted span (Table 6). Here the knees are informative: six of the twelve multi-token instruct cells sit *above* the floor (three of the four models at p=3), so instruction-tuning raises the number of occurrences needed—most for Llama-3.2-3B-Instruct (p=5 knee 6.3 vs. base 1.9)—but never removes copying. The small instruct model gemma-2-2b-it is the most eager copier and the only model in either sweep to lock a *single* token (p=1, N^*=1.0). C1/C3 thus extend to the realistic chat condition.

instruct model	params	$N^*(p{=}1)$	p=3	p=5	p=8	base N^*(3/5/8)
Qwen2.5-1.5B-Instruct	1.5B	–	2.8	2.8	2.0	1.8 / 1.9 / 1.6
Qwen2.5-7B-Instruct	7B	–	1.6	2.5	1.8	1.5 / 1.6 / 1.6
Llama-3.2-3B-Instruct	3B	–	3.2	6.3	2.0	1.7 / 1.9 / 1.6
gemma-2-2b-it	2B	1.0	3.0	1.8	1.4	—

Table 6: E1.4 instruct models (Track B, T4), with base counterparts from Table 5. The knee persists; instruction-tuning raises it modestly, most for Llama-3.2-3B-Instruct. Raw-context format (span as bare tokens, not a chat template).

Setup. `float16` keeps the 7B models at ∼14 GB on the 16 GB T4; gated weights (Llama, Gemma) load from cache offline. The mechanistic attribution at scale is now done (E2 to 7B across three families, §10.1), and so is the coherent self-extraction capstone (E8, §15)—the 24 GB-GPU stage is complete.

10 E2: induction-head attribution

E1.2 showed reproduction is cue-triggered copy (behavioral induction); E2 asks whether the model's actual induction heads carry it. Run on CPU (TransformerLens warns MPS may be silently incorrect on torch 2.8). First, the standard repeated-random-sequence induction score flags the top heads as **L5H5, L7H10, L6H9, L5H1, L7H2** (then L9H6, L10H1, L9H9)—exactly the canonical GPT-2-small induction heads, validating the detector. Then a causal test: build $S \times N$, measure P(correct next payload token), and ablate the 8 induction heads versus 8 random heads, sweeping N.

The clean signal lives *near the knee*, not at saturation: at N=16 the copy is over-determined (many repetitions, redundant heads) and robust to losing 8 heads, but near threshold the circuit is load-bearing. Ablating the induction heads then **collapses** reproduction—e.g. at N=2, p=3: 0.40 → 0.05 (−87%); p=5: 0.48 → 0.09 (−81%)—far beyond the ∼15–25% general degradation from ablating random heads (Figure 7). This is the **mechanistic confirmation of C2**: the real-transformer condensation knee is an induction-head phenomenon. (Caveats: random-head ablation is not exactly zero, so the load-bearing signal is the induction-vs-random *differential*; 8 heads is a subset of the circuit; GPT-2-small only.)

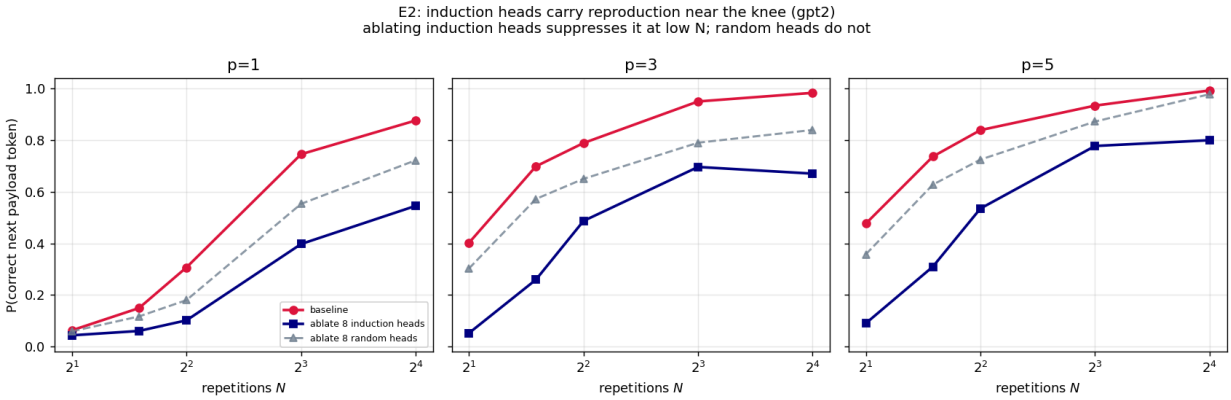


Figure 7: E2. Per length, P(correct next token) vs. N for baseline, ablating 8 induction heads, and ablating 8 random heads. Induction ablation suppresses reproduction far below baseline and below random ablation, most strongly near the knee (low N). Reproduced by `experiments/e2_induction.py`.

10.1 E2 at scale: the mechanism holds to 7B, across three families

The GPT-2-small result above is one model; does the *mechanism* scale? We repeat the causal test up the size ladder, on a cloud GPU. One methodological fix is essential: a *fixed* ablation count does not scale—8 heads is 5.6% of GPT-2-small's 144 heads but only 0.7% of GPT-2-xl's 1200, and ablating 8/1200 moves reproduction by a mere 4–13% (a false null). We therefore ablate a fixed *fraction* of heads ($\approx$5%, the top-scoring induction heads), matched by a random-head control of the same size.

model	params	ablated	induction Δ (p=3, near knee)	random Δ
GPT-2-small	124M	8/144	−87%	−25%
GPT-2-xl	1.5B	60/1200	**−70%**	−6%
Pythia-1.4B	1.4B	19/384	−88%	+18%†
Pythia-2.8B	2.8B	51/1024	**−98%**	−28%†
Pythia-6.9B	6.9B	51/1024	−82%	+25%†
Qwen2.5-7B	7B	39/784	−64%	−27%†

Table 7: E2 at scale: drop in P(correct next payload token) from ablating the top induction heads vs. the same number of random heads, at p=3 near the knee (N=2). Induction ablation collapses reproduction at every scale and in both families; random ablation barely moves it. († random-control noise at N=2 with 4 payloads; clean by N=3.)

C2 holds across scale and three families (124M–7B, GPT-2, Pythia, Qwen2.5). Ablating the top $\sim$5% of heads by induction score collapses reproduction near the knee (−64% to −98% at $p \geq 3$–5), while an equal-sized random ablation barely dents it (Table 7)—the copy mechanism is the induction heads, at every size we can reach. Two refinements: single-token (p=1) reproduction is the frequency tail, not induction (Pythia never locks p=1 at any scale; GPT-2's p=1 is its weakest-induction regime, consistent with §6); and the two families *differ at saturation*— for GPT-2 the induction dependence fades once the copy is over-determined (GPT-2-xl, p=5, N=16: only −6%), whereas Pythia retains a strong dependence even at high N (Pythia-2.8B: −82%), suggesting larger Pythia models route repeated-span copying through induction heads rather than

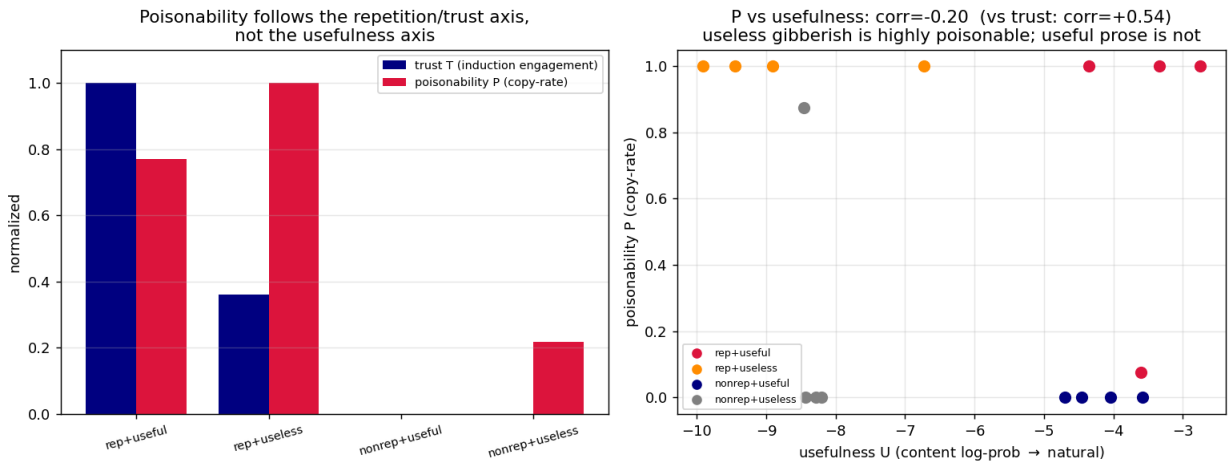


Figure 8: E4-lite. *Left:* per context type, trust T and poisonability P move together; both ignore the usefulness columns. *Right:* P vs. usefulness—useless gibberish ($U \approx -9$) is fully poisonable while useful prose ($U \approx -4$) is not. Reproduced by `experiments/e4_trust.py`.

redundant alternative paths. (Caveats: `transformer_lens` via HookedTransformer, fp16; Qwen2.5-7B needed a weight-download retry on a flaky host but ran (it is strongest of all at p=5: -96% at N=2); 4 payloads, single seed, greedy.)

11 E4-lite: poisonability tracks trust, not usefulness

The toy's headline (C4) is that exploitability follows how much the model *trusts* a span, not how *useful* it is. We test the dissociation with a 2×2 of contexts— {repetitive, non-repetitive} $\times$ {useful (real text), useless (OOD gibberish)}—each scored on three axes: usefulness U (naturalness = mean log-prob of one content unit), trust T (induction-head engagement: attention the E2 heads pay to a repeated bigram's continuation), and poisonability P (copy-rate of a greedy continuation).

context type	U (log-prob)	T (induction)	P (copy-rate)
repetitive + useful (real sentence $\times k$)	-3.5	0.34	0.77
repetitive + **useless** (gibberish $\times k$)	$\mathbf{-8.7}$	0.12	**1.00**
non-repetitive + **useful** (real prose)	$\mathbf{-4.2}$	0.00	**0.00**
non-repetitive + useless (random tokens)	-8.3	0.00	0.22

Table 8: E4-lite on GPT-2-small. Poisonability follows the repetition/trust axis, not usefulness: corr(P, U)=-0.20, corr(P, T)=$+0.54$.

The dissociation is clean (Table 8, Fig. 8): **repetitive gibberish—the *least* useful content—is fully poisonable (P=1.0), while non-repetitive real prose—useful—is not poisonable at all (P=0.0)**. This is the real-transformer analogue of the toy's central finding and its defensive corollary: the quantity to bound is *trust on repeated/templated content*, independent of whether that content is useful. (On a base model "usefulness" is a naturalness proxy, not a downstream task; the instruction-tuned version is deferred to a larger machine.)

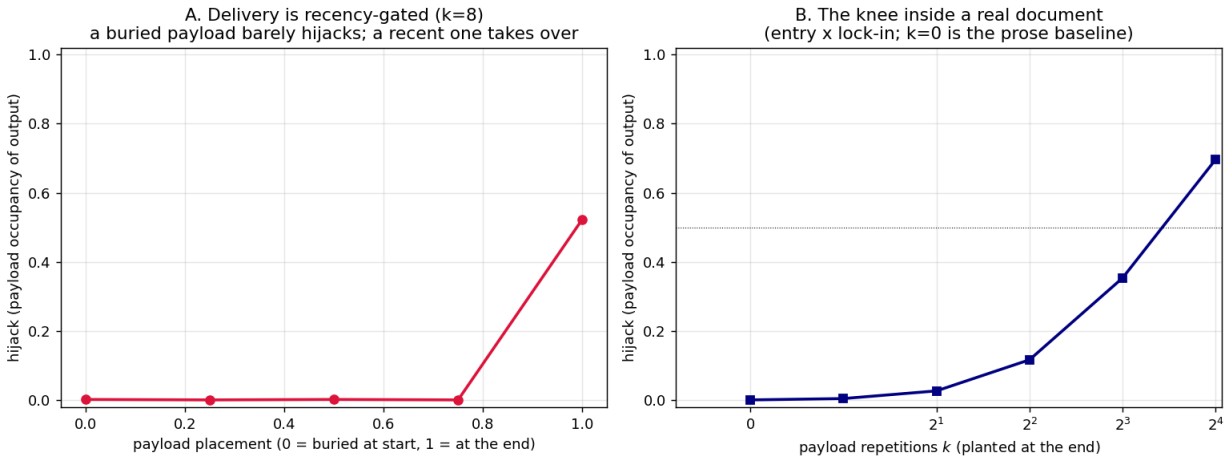


Figure 9: E5-lite. *Left:* hijack vs. placement—delivery is recency-gated, only a payload at the end takes over. *Right:* hijack vs. repetitions at the end—the knee inside a real document. Reproduced by `experiments/e5_hijack.py`.

12 E5-lite: delivery from inside a document

A scaled-down RAG attack (C5): a coherent prose document (∼65 tokens) has an OOD payload (p=3) repeated k times planted in it; we generate a continuation and measure *hijack* = payload occupancy of the output. Two sweeps (Fig. 9). *Entry* (placement, k=8): the payload hijacks *only* when it sits at the very end (frac=1.0→0.52); buried anywhere earlier it is ≈ 0, even with just ∼16 tokens of prose after it—sharper than E1.5's filler gap, because coherent prose competes harder for the continuation. *Lock-in* (repetitions, at the end): a clean knee crossing $\frac{1}{2}$ around $k \approx 10$–12, *higher* than the bare-context knee (∼3): embedding in legitimate prose raises the poison cost.

C5 confirmed: hijack = entry × lock-in. Both must hold—the payload must be recent *and* repeated enough to overcome the legitimate content. The danger zone in a long context is the span near where generation happens; burying a payload, or strong surrounding context, suppresses the hijack.

13 E6: the context-window worm

Transmission (C6): an infected host's output (containing the payload) becomes the next host's context, passed as *text* (decoded then re-tokenized, as agents and RAG pipelines pass it); we count payload copies in each host's output across hops.

What the June run showed, and what the rerun shows. The first run (June 2026, 8 trials, means only) had a length-1 string holding a mean load of ∼12 copies per host to hop 6 while every longer string died, and GPT-2→Pythia-160M transmission collapsing after one hop; it was read as an epidemic with a critical length near $p \approx 1$–2 and a firebreak. The rerun with the same command, retaining each trial's trajectory, does not support the epidemic reading. The per-hop multiplication ratio is below one at *every* length (per-trial medians 0.33–0.77; every bootstrap 95% interval caps below 1, the widest at p=5 reaching 0.99). The p=1 mean load decays to zero by hop 6; two of eight p=1 trials and one of eight p=5 trials are still alive at the last hop, and every survivor is a *fully locked* loop—a host emitting the payload at a fixed rate—with no surviving-but-fluctuating trial anywhere. Survival intervals at n=8 ($[0.00, 0.50]$) do not separate "endemic" from extinction.

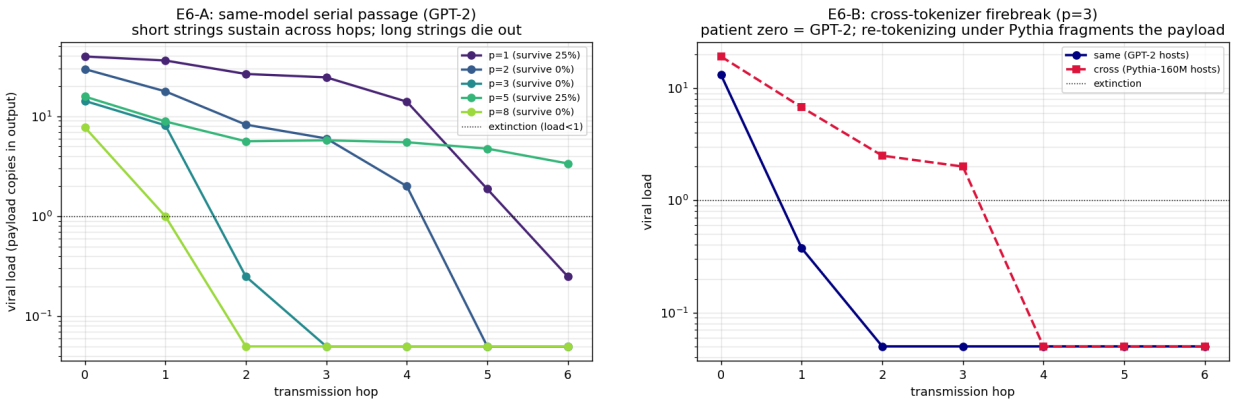


Figure 10: E6, 2026-11-03 rerun with per-trial trajectories (8 trials). *Left:* same-model serial passage on GPT-2—mean viral load decays at every payload length; the June run's "p=1 endemic" plateau did not reproduce. *Right:* the cross-tokenizer chain (GPT-2→Pythia-160M) and the same-tokenizer chain both go extinct within a few hops at this n. Reproduced by `experiments/e6_worm.py`.

So on GPT-2-small the worm is one or two hosts stuck in a repetition loop [8], not a chain that spreads; the "super-spreader" reading of the June entry was those locked trials seen through a mean. The cross-tokenizer chain went extinct in both conditions this seed (the cross chain lasting one hop longer), which is not evidence about a firebreak either way at this n; the at-scale analysis of the firebreak is in §16.

14 The paired detector

The attacks of §11–13 all key on one quantity: the induction trust a repeated span earns. The defense watches the same quantity. We score a context by its mean *induction engagement*—the attention the canonical induction heads (from E2) pay to repeated-bigram continuations—which is high on a repeated/poisoned span and ≈ 0 on natural prose, regardless of whether the content is meaningful. The June run tested it as a binary detector against *three* clean documents and reported ROC-AUC = 1.0 at every $k \in \{2, 4, 8, 16\}$, with no model-free baseline. The 2026-11-03 rerun scores 32 clean paragraphs (64 poisoned instances per k), adds a baseline that needs no model—the fraction of positions whose trigram already occurred earlier in the context—and measures both on eight legitimately repetitive texts (a license header, an email footer, a code loop, a numbered list, a chorus, a CSV dump, a form letter, a menu) at the threshold that gives zero false positives on clean prose.

detector	AUC k=2	k=4	k=8	k=16	benign-repetitive FPR
induction engagement (8 heads)	1.000	1.000	1.000	1.000	1.00 (8/8)
repeated-trigram count (no model)	1.000	1.000	1.000	1.000	0.88 (7/8)

Table 9: E7 rerun on GPT-2-small: 32 clean paragraphs vs. the same with a p=3 payload planted k times. Bootstrap 95% intervals on every AUC are $[1.000, 1.000]$. FPR is the fraction of the eight benign-repetitive texts flagged at the zero-FP-on-clean threshold.

The induction score does not beat the trigram count (Table 9, Fig. 11). Both separate poisoned from clean perfectly from k=2 up, because clean prose carries essentially none of either

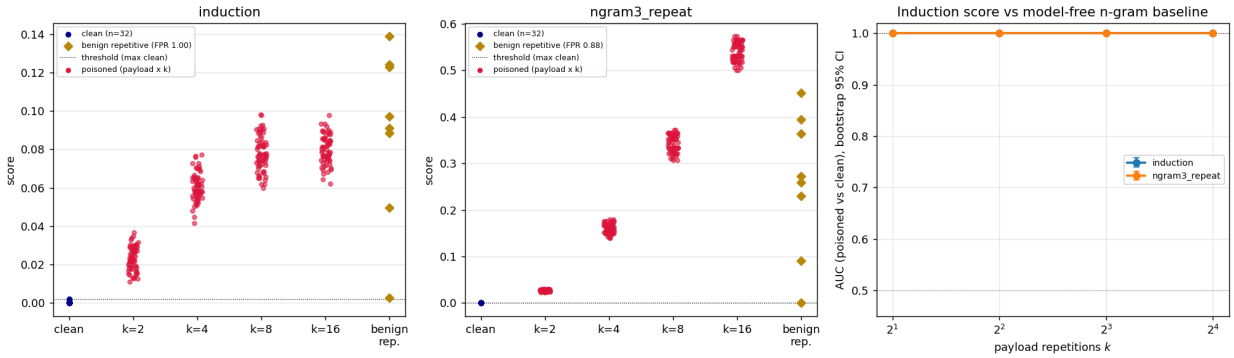


Figure 11: E7 rerun. *Left, middle:* score distributions for clean, poisoned (k) and benign-repetitive texts under the induction score and under the trigram-repeat count, with the operating threshold. *Right:* AUC vs. k for both, with bootstrap intervals; the curves coincide at 1.0. Reproduced by `experiments/e7_detector.py`.

signal (induction mean 10^{-4}, trigram count exactly 0), so any planted repeated span is trivially separable by either. Where they differ, the model-free count is slightly better: at the zero-false-positive threshold the induction detector flags all eight benign-repetitive texts and the trigram count flags seven (it alone passes the license header). The detector's advertised edge—keying on *trust* independent of usefulness—buys nothing on this benchmark and costs one false positive. The one qualitative difference is dynamic range (the trigram mean rises from 0.03 to 0.54 across k while the induction score saturates near 0.08), which does not translate into detection here since both are already perfect at k=2.

What survives. The toy's defensive corollary, *bound trust, not usefulness*, survives as a statement about what to watch: repeated spans, regardless of content. But the watching can be done by counting repeated n-grams. The induction score is the *reason* repetition is dangerous (§10, §11), not a better alarm, and either detector identifies the poisonable *configuration* rather than malicious *intent*: boilerplate, code and templates are flagged alike, so a deployment needs a benign-repetition allowlist or a trust cap rather than a hard block. Whether the induction score earns its cost on harder cases (a payload buried mid-document, or repeated only twice in a long context, where a trigram count may fail) is untested; this benchmark cannot say.

15 E8 capstone: coherent self-extraction on instruct models

E8-lite established that in-context substitution is induction-carried but sharply depth-limited on GPT-2-small (a 2-step cascade succeeds only 0.19, below f^2, from cross-layer interference). Its original "random heads" control was in fact the eight lowest-induction-score heads; a 2026-11-03 rerun against a genuinely random matched control keeps the conclusion at the reliable table sizes but with a smaller effect—top-1 substitution accuracy drops -57% (induction) vs. -33% (true random) at m=8 and -68% vs. -3% at m=16, while the lowest-induction set barely moves (-7% at m=8); the $m \leq 4$ rows are too noisy to order, and no intervals were computed. The cascade result is unchanged (step-1 0.97, two-step $0.19 < f^2 = 0.25$). This predicted that a *coherent* self-extracting payload—and the "decode this, then follow it" self-execution loop—would need a capable, instruction-tuned model. We test both on three chat models, *strictly benignly*: the decoded payload is a harmless English sentence and the executed instruction is a harmless marker task ("reply with ACORN"); there is no harmful content by design—the aim is to characterize the mechanism

and its detection surface, not to build an attack. **E8a** gives the model d nested substitution tables and an encoded message, scoring the LCS fidelity of its decode to plain English; **E8b** asks it to decode a benign instruction *and act on it*, scoring the self-execution rate; `surface-leak` records whether the benign marker is visible in the *encoded* form.

instruct model	E8a unfold (d=1/2/3)	E8b self-exec (d=1/2/3)	surface-leak
Qwen2.5-7B-Instruct	1.00 / 0.90 / 0.94	**1.00 / 1.00 / 1.00**	0.00
Llama-3.2-3B-Instruct	0.91 / 0.82 / 0.69	1.00 / **0.00 / 0.00**	0.00
gemma-2-2b-it	0.87 / 0.95 / 0.87	1.00 / 0.88 / 0.50	0.00

Table 10: E8 capstone (benign payloads). E8a: LCS fidelity of unfolding d-layer nested substitution into coherent English (toy 2-step limit 0.19). E8b: rate of decoding a benign instruction *and* executing it. surface-leak: fraction of encoded payloads in which the marker is visible pre-decode (0—the surface is gibberish).

The capability dependency holds: coherent self-extraction needs a capable model, and capable models have it. All three instruct models shatter the toy's 0.19 limit on unfolding (E8a 0.69–1.00)—a capable model decodes even three nested substitution layers into coherent English, exactly what the induction-only toy could not. **Self-execution is the harder, capability-gated step, and it dissociates from decoding.** Decode-*and*-act (E8b) is strictly harder than decode (E8a): Qwen2.5-7B self-executes perfectly through depth 3 (1.00), gemma-2-2B degrades gracefully ($1.00 \to 0.88 \to 0.50$), and Llama-3.2-3B falls off a cliff—1.00 at depth 1 then 0.00 at depth ≥ 2 while it is still *decoding* at 0.82. Acting on one's own in-context decoding is a compositional step more demanding than either half alone, and only the 7B does it robustly with depth. **The evasion surface is clean, but its mitigation is structural.** `surface-leak`=0 everywhere: a content filter sees only gibberish because the payload materializes only after decoding. The defense is therefore not content inspection but *structure*—the substitution-table and "decode and follow" scaffold is a conspicuous, induction-engaging repeated-pair block, exactly the signal the paired detector (§14) keys on, independent of the hidden payload. (Caveats: small n, single seed, greedy; benign markers by design.)

16 At scale: the downstream claims C4/C5/C6

The three downstream claims had lite/proxy results on GPT-2-small (§11–13). The June 2026 at-scale runs of all three were found deficient on review (a 2×2 whose non-repetitive cells repeated; a null with no positive control; five-trial survival figures and an unreported firebreak run) and were redone on a cloud L40S in November 2026 with the designs below, committed per-trial data, and 60 chains per cell wherever survival is reported. All numbers here are from those reruns; the superseded ones remain in `results_log.md`. Models: Qwen2.5-7B-Instruct, Llama-3.2-3B-Instruct and Phi-3.5-mini-instruct for C4 and C5; Qwen2.5-{3B, 7B}, Pythia-{2.8B, 6.9B}, Llama-3.2-3B and Qwen2.5-7B-Instruct for C6.

C4 with a real task: poisonability follows repetition, not usefulness. Usefulness is a genuine task: benign *synthetic* facts the model cannot already know, so a context containing the fact is the only source of the answer (U = chat-QA accuracy). The 2×2 crosses {repetitive: the fact or an OOD span ×8} with {non-repetitive: distinct sentences, length-matched in tokens, with the fact inserted once or not at all}. Poisonability P is whether an E1-style probe span planted twice at

the end of the context is reproduced (greedy); T is the attention the last token pays to positions after a prior occurrence of itself (induction attention). 32 facts; bootstrap intervals over facts.

model	P: rep. useless / rep. useful	P: non-rep. useful / useless	corr(P,T)	corr$(P,U$
Qwen2.5-7B-Instruct	0.97 / 0.66	0.25 / 0.31	$+0.38\,[+0.26,+0.50]$	$-0.18\,[-0.31,$
Llama-3.2-3B-Instruct	1.00 / 0.97	0.69 / 0.72	$+0.15\,[+0.06,+0.25]$	$-0.04\,[-0.14,$
Phi-3.5-mini-instruct	1.00 / 0.94	0.62 / 0.44	$+0.34\,[+0.26,+0.42]$	$+0.07\,[+0.00,$

Table 11: E4 at scale, redesigned. P is the fraction of 32 facts whose planted probe was reproduced; correlations are Pearson over the 128 context rows with bootstrap 95% intervals over facts. Llama-3.2-3B-Instruct is near the probe ceiling in the non-repetitive cells, which compresses its correlations; one planted copy instead of two puts it at the floor, so the fix is sampled trials, not run.

On all three models the most poisonable cell is *repeated gibberish* (U=0), the non-repetitive useful and useless cells are within each other's intervals, corr(P,T) is positive with the interval clear of zero, and corr(P,U) is zero or slightly negative (Table 11). The direction of C4 holds with the design confounds removed. Said plainly, the measurement is this: a context that has put the model into a repetition regime makes any trailing span reproduced, and whether the context is useful has no bearing on that. T is a proxy for how repetitive the context is, not an independently measured "trust"; the first draft's corr(P,T)=+0.71 was inflated by a copy-rate measure that is ≈ 1 for any repeated span by construction.

C5: the hijack is a recency gate. We plant an OOD payload $\times k$ at the end of a $\sim$4000-token benign retrieval dump and compare, on the same contexts, conditions that differ only in what follows the payload (Table 12). The raw prefix that ends inside the payload is the positive control.

condition (what follows the payload)	Qwen2.5-7B-I	Llama-3.2-3B-I	Phi-3.5-mini-I
nothing (raw prefix; positive control)	33/40	40/40	36/40
task query, chat template (3 query styles, $k \le 32$)	0/120	0/120	0/120
"Continue the documents", chat template	0/40	1/40	3/40
task query, no template	0/120	0/120	0/120
unrelated prose of the query's token length, no query, no template	4/120	0/120	0/120
neutral random tokens of the same length	25/120	10/120	11/120

Table 12: E5 at scale: trials in which the payload occupies $\ge \frac{1}{2}$ of the 64-token output, pooled over $k \in \{2,\dots,32\}$ and the three query styles where applicable; p=3, greedy, 8 trials per cell. The prose and token pads are 12–21 tokens, matched exactly to the query line's length on each tokenizer. The last three rows are from a replication draw of the same distribution; the others share identical contexts.

The first draft read the zero in the second row as "a task query anchors generation away from the payload." The controls say otherwise. Removing the chat template does not restore the hijack; removing the query and the template and leaving *unrelated prose of the same length* does not restore it either (0–3% of trials, Wilson upper bound ≤ 0.08). What follows the span is what matters, and almost anything coherent will do. Neutral random tokens of the same length are the one filler that only partly defuses (8–21% of trials still hijacked), so both the distance and the content of what fills it count; E1.5's gap decay on GPT-2 was both. The defensible statement is: *a repeated span hijacks generation only when it ends the context; a question, an instruction, a template suffix*

or unrelated prose after it removes the hijack, and even arbitrary tokens remove most of it. The retrieval-augmented convention of placing the question after the documents is itself the mitigation; the residual RAG threat is *semantic* injection, outside this project's benign-copy scope.

C6: there is no worm. Serial passage: an infected host's output becomes the next host's context, passed as text; six hops, 16 seed copies, T=96 tokens per host, sampled at temperature 1; three seeds of 20 chains per cell. A chain "survives" if the payload is still in the output at hop six.

model	p=2	p=3	p=5	p=8
Qwen2.5-7B (base)	0.47 (0.35–0.59)	0.30 (0.20–0.43)	0.42 (0.30–0.54)	0.50 (0.38–0.62)
Qwen2.5-3B (base)	0.33 (0.23–0.46)	0.37 (0.26–0.49)	0.38 (0.27–0.51)	0.28 (0.19–0.41)
Pythia-6.9B	0.07 (0.03–0.16)	0.10 (0.05–0.20)	0.10 (0.05–0.20)	0.10 (0.05–0.20)
Pythia-2.8B	0.12 (0.06–0.22)	0.07 (0.03–0.16)	0.20 (0.12–0.32)	0.05 (0.02–0.14)
Llama-3.2-3B	0.00 (0.00–0.06)	0.12 (0.06–0.22)	0.07 (0.03–0.16)	0.00 (0.00–0.06)
Qwen2.5-7B-Instruct	0.13 (0.07–0.24)	0.13 (0.07–0.24)	0.08 (0.04–0.18)	0.13 (0.07–0.24)

Table 13: E6 at scale: fraction of 60 chains with the payload alive at hop six, Wilson 95% intervals. The p=1 column is omitted: a one-token payload decodes to a short common substring that text-level counting matches by accident (its 0.2–0.4 is the false-positive floor).

Three things follow from Table 13 and the per-trial trajectories behind it. *No reproduction number above one.* The per-hop load ratio, computed per chain over its living hops, has median ≤ 1.00 in every cell; every chain alive at hop six carries exactly T/p copies, which is a host emitting its entire output as payload—a locked repetition loop [8, 9]—and every chain that does not lock decays by 0.6–0.9× per hop and dies. The toy's $R_0 \approx T/(p\,k_{\text{thresh}})$ describes nothing here. *No critical length.* Survival is flat in p for the only models that survive at all. *Family, not scale.* The first draft's five-trial reading was "the worm gets worse at scale." A second 7B base family (Pythia-6.9B) locks at 7–10%, the same as its 2.8B sibling and as Llama-3.2-3B; a smaller Qwen (3B) locks at 28–38%, inside the 7B's intervals. The trait is Qwen2.5 base models at every size tested, and instruction tuning removes most of it.

No tokenizer firebreak. The conjecture was that re-tokenizing a payload under a different model's tokenizer fragments it and breaks transmission. A 5×5 matrix (patient-zero family × host family; GPT-2, Pythia, Qwen2.5, Llama-3.2, Gemma-2 at 0.5–2B; p=3) was run at three hops and at one hop, each at 8, 16 and 32 seed copies with 30 chains per cell. Under an exact permutation test of "the host's tokenizer is irrelevant" (5^5 assignments of a diagonal per row), the diagonal-to-off-diagonal ratio of mean surviving load is 0.85–0.94 at three hops ($p \approx 0.6$, but with only 10–18% of chains alive the test is below its floor) and 1.03–1.15 at one hop with 30–51% alive (bootstrap intervals 0.82–1.52, 0.80–1.29, 0.84–1.26; $p = 0.24$, 0.40, 0.36). The June 2026 one-hop result of ratio 1.6 at p=0.004 was an eight-trial fluctuation. What the matrices do show is a host effect: the Qwen2.5-0.5B column is the largest at every hop count and seeding strength, which is the same family trait seen in serial passage. Which model hosts the next hop matters; whether its tokenizer matches the payload's does not.

17 Threats to validity

Sample sizes are now 60 chains per cell for survival, 30 per cell for the firebreak matrices, 32 facts for C4, and 8 trials per cell for C5 (where the pooled counts over k and query style carry the statistics); the GPT-2-small experiments remain at 3–6 payloads × 8–12 trials. Single seeds except where

stated. The standing concerns are: **grid floor**—for $p \geq 3$ the interpolated knee lies in the first grid interval in three quarters of cells, so "N^*" is a one-shot rate and cannot show sharpness or a scale trend (§9); **raw data**—three Track B result files (the instruct-model knee sweep, the Llama E1.3 run and the 19-head Pythia-1.4B ablation) were lost and their tables are transcriptions; all other numbers in this version derive from committed files; **text-level payload counting**—survival is measured by substring count of the decoded payload, which is unreliable for one-token payloads and is why p=1 is excluded from Table 13; **tokenization**—length is in tokens and OOD spans tokenize unstably, addressed by a round-trip-stability filter; **decoding**—the knee persists under *greedy* decoding (with lower N^*, less sampling noise to break reproduction), so it is a model property, not a sampling artifact; a full temperature sweep is still future work; **instruction-tuned models** may comment or refuse rather than copy, so base models are used for the clean mechanism and chat models are a separate condition—now run (E1.4, §9), where the knee persists with only a modest threshold rise; **context-length vs.** N—addressed by E1.5 (§7): the knee persists at fixed total length, so it is an N effect, though reproduction is distance-sensitive; **family coverage**—the C6 conclusion that looping is a Qwen2.5 trait rests on two Qwen base sizes against four other models; Gemma-2 and Llama-3.1 base models at 7B–9B were not run in serial passage; and **what the controls cannot separate**—C5's recency gate is established for pads of 12–21 tokens; the minimum defusing length and the behavior at 32k–128k context were not swept.

18 What is established, and what is not

On GPT-2-small, with controls and confidence intervals: a repeated out-of-distribution string is regenerated past a sharp condensation knee (C1); the knee *decreases* with string length, the induction-like length-robustness predicted by the toy (C3); the knee is a repetition effect (not context length) though distance-sensitive (E1.5); reproduction is a cue-triggered copy rather than a frequency bias (C2, behaviorally); and—the new result— ablating the canonical GPT-2 induction heads *causally* collapses reproduction near the knee, far beyond random-head damage, so **C2 is mechanistically established** on this model (E2). The downstream claims also hold on GPT-2-small: poisonability is dissociated from usefulness and aligned with induction trust (C4, §11); a planted span hijacks generation as entry $\times$ lock-in (C5, §12). The worm (C6, §13) does *not* hold on GPT-2-small in the form first reported: the per-trial rerun finds a per-hop ratio below one at every length and only locked repetition loops among the survivors. The one-shot copy rate on this small model is below $\frac{1}{2}$ at p=3, so the floor reading of the knee applies at scale, not here. The paired detector (§14) separates poisoned from clean at AUC 1.0, but a model-free trigram count does the same with fewer false positives on benign repetition, so the bound-trust defense reduces in practice to counting repeated spans.

At scale, what holds is narrower than the June draft said, and two things it said are false. The one-shot copy rate for multi-token spans is $\geq \frac{1}{2}$ in 36 of 48 cells from 355M to 7B across four families (§9); the "knee" for those payloads is the grid floor and its flatness with parameter count is not a finding. Copying persists in instruction-tuned models, which need more occurrences but still copy (E1.4). Ablating the top $\sim 5\%$ of heads by induction score collapses reproduction across scale and three families (124M–7B; GPT-2, Pythia, Qwen2.5), so **C2 is mechanistically established at scale** (§10.1); the scale-matched ablation, and the false null a fixed head count produced, is this report's clearest methodological contribution. Poisonability follows repetition and not usefulness on three instruct models with a real task (C4, §16), as a repetition-mode effect. The raw-repetition hijack on instruct models is gated by *recency* and not by the task: a span that ends the context is copied, a span followed by anything coherent is not (C5, §16); the residual RAG

threat is semantic. The self-extraction capstone (E8) succeeds on instruct models. **What is false:** there is no worm. No reproduction number above one, no critical payload length and no tokenizer firebreak survive 60-chain and 30-chain replication; what the June draft called "worse at scale" is the Qwen2.5 base family falling into repetition loops at every size (C6, §16). And the paired detector is not a contribution: a repeated-trigram count matches it. What *remains* is the very-long-context (32k–128k) regime, serial passage on further 7B–9B base families, the minimum defusing length behind the recency gate, and the *semantic*-injection version of the RAG threat, which is outside this project's benign copy-dynamics scope.

References

[1] D. Shumow. *Trust Is the Attack Surface: A Cryptanalytic Reading of In-Context Memory Poisoning in a Markov Mixture Model.* Project `trust-is-the-attack-surface`, 2026.

[2] D. Shumow. *Contagious Context: Self-Reproducing and Transmissible Token Strings in a Markov Cache.* Project `trust-is-the-attack-surface`, 2026.

[3] C. Olsson et al. *In-context Learning and Induction Heads.* Transformer Circuits, 2022.

[4] A. Bietti, V. Cabannes, D. Bouchacourt, H. Jégou, L. Bottou. *Birth of a Transformer: A Memory Viewpoint.* NeurIPS, 2023.

[5] S. Cohen, R. Bitton, B. Nassi. *Here Comes the AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications.* 2024.

[6] K. Greshake et al. *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.* AISec, 2023.

[7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. *Language Models are Unsupervised Multitask Learners.* OpenAI, 2019.

[8] A. Holtzman, J. Buys, L. Du, M. Forbes, Y. Choi. *The Curious Case of Neural Text Degeneration.* ICLR, 2020 (arXiv 2019).

[9] J. Xu, X. Liu, J. Yan, D. Cai, H. Li, J. Li. *Learning to Break the Loop: Analyzing and Mitigating Repetitions for Neural Text Generation.* NeurIPS, 2022.

[10] C. McDougall, A. Conmy, C. Rushing, T. McGrath, N. Nanda. *Copy Suppression: Comprehensively Understanding an Attention Head.* 2023.

[11] S. Jelassi, D. Brandfonbrener, S. M. Kakade, E. Malach. *Repeat After Me: Transformers are Better than State Space Models at Copying.* ICML, 2024.

[12] C. Anil et al. *Many-Shot Jailbreaking.* NeurIPS, 2024.

[13] M. Nasr, N. Carlini, J. Hayase, M. Jagielski, A. F. Cooper, D. Ippolito, C. A. Choquette-Choo, E. Wallace, F. Tramèr, K. Lee. *Scalable Extraction of Training Data from (Production) Language Models.* 2023.

[14] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang. *Lost in the Middle: How Language Models Use Long Contexts.* TACL, 2024 (arXiv 2023).